

+5,449 -4,141

AI and Good Code

Software Engineering and The Day After Tomorrow

AI Is Amazing

- Knows *all* the languages, idioms, libraries
- Explains unfamiliar code better than I do
- Generates working code quickly
- Kinda feels like it can do my job?

AI Slop Is Terrible

- Huge diffs nobody wants to review
- Code that works but is full of workarounds
- Tests that prove less than they claim
- Maintainers pushing back

How can both be true?

AI and Good Code

What do you bring to the table when AI writes the code?

What is good code?

Where We're Going

- The fast path to fragile code
- Why we care about code structure
- Steering AI toward durable code

The Fast Path to Fragile Code

The Fast Path

Day 1

- Great idea for a feature
- Ask AI to build it
- It writes a lot of code
- It works
- You feel fast

Day 2

- Edge cases appear
- The code gets longer
- You understand less of it
- AI keeps getting through it
- You keep going

Day 3

- The feature works
 - The edge cases seem handled
 - ...but others may be lurking
- `+5,449` `-4,141`
 - You can't really review it
 - ...Neither can anyone else
- It doesn't get merged.

What happened?

What Happened?

AI is great at writing code. It's not great at *living with* it.

- We learned to write code this way in school:
solve the problem just well enough. Move on.
- AI is trained the same way. Make it work. Move on.
- It doesn't have to maintain the codebase three years later.

AI GOAL

**"Make it
Work"**

YOUR GOAL

**"Make it
Good"**

Why We Care About Code Structure

Software engineering is the discipline of building interfaces.

Do we know how to maintain code?

Yes. (Mostly.)

- That's what software engineering *is* as a discipline: building **interfaces**.
- The boundaries that let humans collaborate on code over years.
- Types, modules, packages, filesystems, processes, protocols. We didn't always have these.

A Thought Experiment

Imagine all we had was **assembly** and **raw disks**. Read bytes from memory. Write bytes to disk. That's it.

Over decades, we built **programming languages** with type systems and modules. And **filesystems** with directories, ownership, and permissions.

Would AI create these abstraction layers?

No. Not on its own. **But with your help, it can.**

What Is Good Code?

Good code is **maintainable** and **extendable**.

Good code lets you change one thing without reading everything.

Do we need this with AI?

"AI can read everything. Aren't abstractions less important now?"

No. I think they matter more.

- AI lets you get further with a sloppy codebase than humans alone could. For a while.
- But AI also has a finite context window, and tokens cost money.
- Better abstractions help **both humans and AI** do their best work.

The Scope of the Argument

Disposable Code

Prototypes, internal scripts, vibe-coding, days to weeks of shelf life.

Let AI run wild.

Durable Code

Product logic, libraries, collaborative repos, open-source, years of shelf life.

Engineering is required.

The rest of this talk is about **durable code**.

Steering AI Toward Durable Code

What **you** bring to the table

What You Bring

Business context and experience.

- What does the business need?
- Is this a prototype, or production?
- Reliability, or velocity?
- What's the cost of a breaking change?
- How many pages did we get last month? For what?

No LLM has access to that.

Your Role

→ Ensure it's Good Code.

- PR reviews exist because generated code still needs ownership.
- "It passes tests" is not the same as "we can maintain it."

→ Choose the right abstractions.

- Humans invented languages, filesystems, modules, APIs.
- Your codebase needs the same kind of boundary-setting.

→ Live with the consequences.

- AI will generate 8-argument functions, leaky abstractions, workarounds.
- AI won't debug this in three months. You will.

Turn Friction Into Instructions

Friction means **you know something the AI doesn't.**

→ "Keep this PR reviewable; do not rewrite unrelated files."

→ "Preserve the public API unless we explicitly decide to migrate it."

→ "This is a sensitive prod path; prefer boring code and clear failure modes."

Explicit constraints beat implied preferences.

Ask the AI About Code

AI is very good at **understanding and explaining** code. I lean on this constantly.

→ I used to ask *"what is this doing? Why do we need this?"* in code reviews.

→ Now I just ask the AI. Usually it's faster than asking the author.

→ I do this while writing, while reviewing, while debugging.

This is the easy win. AI is a force multiplier for understanding code that already exists.

Guide the Structure

AI can talk fluently about good structure. Ask it to apply that knowledge while planning, building, and reviewing.

Use named principles as working lenses:

- **Make Invalid States Unrepresentable**: encode rules in types.
- **Parse, Don't Validate**: turn strings into structured values once, at the edge.
- **Functional Core, Imperative Shell**: keep logic pure; isolate side effects.
- **Co-locate Decisions with Data**: the code that decides should sit next to what it decides about.

The strange part: AI can explain these principles fluently, then ignore them in the next diff.

I Asked for a Feature

ME

Perhaps we should make the text sizes larger, and some more space between header and content

CLAUDE

```
/* style.css */
.slidev-layout { font-size: 1.35rem }
.slidev-layout p { font-size: 1.4rem }
.slidev-layout li { font-size: 1.4rem }
```

WHAT I CHECKED

Slide Title

- Bullet one
- Bullet two
- Bullet three

ELSEWHERE

Slide Title

A subheader smaller than the bullets

- Body text bigger
- Than its own subheader

I Asked About the Code

ME

```
class="text-xl text-slate-300 mb-4"
```

What are all those class things doing? The text seems to be the same size as the list items.

What is a rem?

CLAUDE

Those are UnoCSS utility classes:

→ `text-xl` → `1.25rem`

→ `text-slate-300` → light gray

→ `mb-4` → `margin-bottom: 1rem`

`.slidev-layout p` has specificity `(0,1,1)`.

`.text-xl` is `(0,1,0)`. The global rule wins; `text-xl` silently loses.

This is a self-inflicted wound from how I wrote the CSS.

`rem` = "root em". Relative to the root element's font-size. By default `1rem` = 16px. UnoCSS's whole text scale is multiples of that root.

I Asked About the Design

ME

What's the idiomatic way of managing this?

CLAUDE

The idiomatic slidev pattern has three layers:

1. **Tune the canvas, not individual sizes.**
`canvasWidth` rescales everything proportionally. The lever I should have reached for first.
2. **Use utilities inline for emphasis.** Don't override base element rules.
3. **Define UnoCSS shortcuts** for repeated patterns: `text-lead`, `text-section-sub`.

```
- canvasWidth: 1280  
+ canvasWidth: 980
```

Smaller canvas, everything else is bigger!

I Asked About the Layers

ME

Have we balanced well across UnoCSS, style.css, and `:where` setups? Where are they working, where are they fighting, where are they at the wrong level?

CLAUDE

A few frictions:

1. `.font-handwritten` lives in CSS but should be a UnoCSS shortcut.
2. Speech-bubble shortcuts duplicate everything except border color. DRY them.
3. Redundant `color` on nested `<li>` – dead since unifying body color.

... 80 lines ...

AI won't clean up or refactor unless you ask.

Ask it to think about *what* to clean up or restructure. Then ask it to do so.

The 3-Step Work Cycle

1

Ask About Structure

Before implementing, sketch the abstraction layers and code boundaries. Use Local Reasoning and correct-by-construction types as guiding principles.

← *Perhaps...
a Claude Skill?*

2

Ask for Options

The `foo_bar_baz` function has a lot of arguments, mostly primitives. What are some ways we could refactor that to produce better code structure?

3

Pick, Then Delegate

Let's go with (b), but create a newtype for `Baz` to encapsulate its behavior.

I Have a Claude Skill for This

```
---  
name: code-structure  
description: Identify design tensions in code structure  
---
```

Goal: code that is easier to read, harder to misuse,
and more resilient to change.

Apply only the principles that matter:

- Make Invalid States Unrepresentable
- Parse, Don't Validate
- Functional Core, Imperative Shell
- Co-locate Decisions with Data

For each tension:

- What's happening?
- Why does it matter?
- Is the fix obvious, tradeoff-heavy, or constrained?

It gives the AI a job, a vocabulary, and a shape for the answer.

Closing

While You Work With AI

- Can I explain why this structure exists?
- Is the PR small enough to review the important decisions?
- Can a future change happen locally?
- Did I give the AI the constraints it needed?

Thank You!

... Questions?